EXATHON INDIA 2026 · TRACK 03

OEE Command Center

Five factory problems. One database. Every answer computed live in Exasol.

Mrinal Prakash Desai · solo · github.com/MrinalDesai/exasol-oee-command-center

THE PROBLEM, IN ONE SENTENCE

A maintenance manager running three plants can’t stop a breakdown before it happens, because the warning signs, the spare parts and the supplier price lists are scattered across systems that each describe the same machine and part differently.

If this worked, he would know which fault is coming hours before the line stops, with the right part already on its way from wherever it sits.

Who: plant and maintenance managers at mid-sized Indian manufacturers: machine shops, auto-component makers, packaging lines.

Five problems, one plant group

Track 03 problem	What it looks like on the shop floor today
Data delayed, distributed, incomplete	Five systems name the same machine five ways, with two clocks, mixed units, late energy data and missing minutes
Unplanned downtime, no clear root cause	The line stops; the team guesses bearing, coolant, overload or drive and works through them one by one
Spare parts in the wrong location	“Out of stock” at the plant, while another store has it under a differently written part number
Quality caught at the end of the line	Bad parts are found at inspection, after the shift has already made them
Suppliers in inconsistent formats	Three price lists, three formats, three currencies or units; none uses our part number

THE APPROACH

Combine first.
Let the pattern name the cause.
Keep the work where the data is.

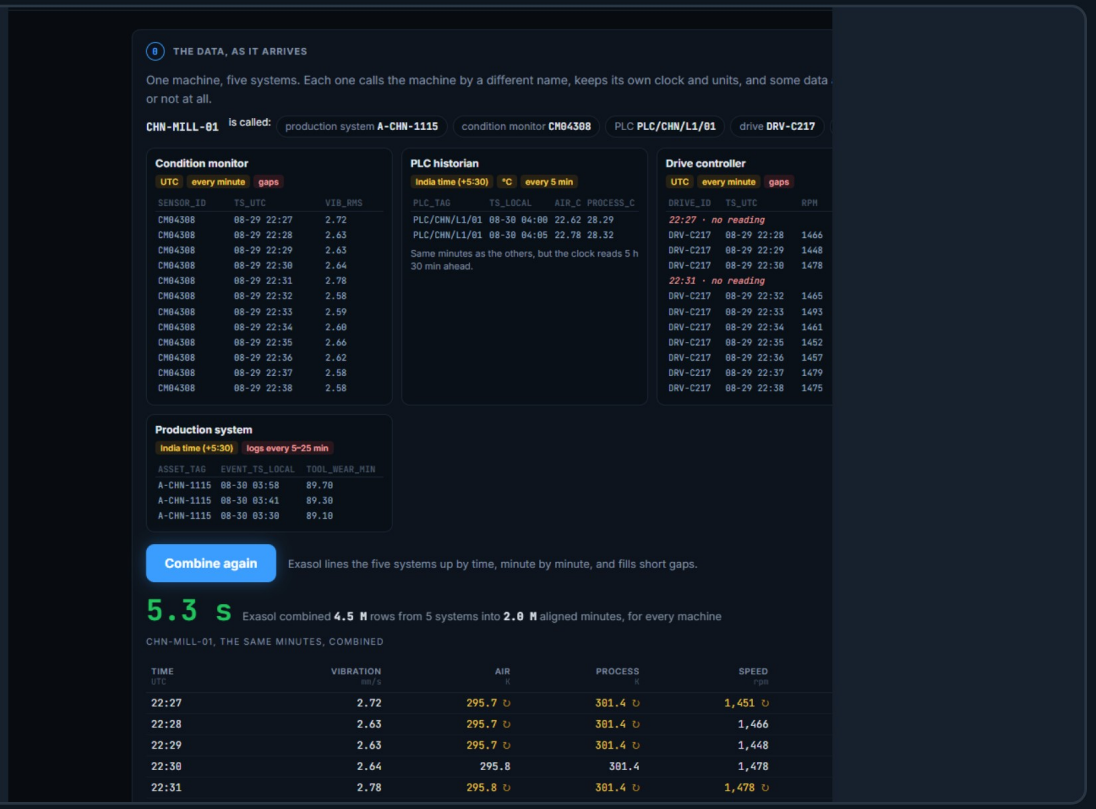
No model sees raw sources: Exasol aligns them first.

No single stream names a fault; the combination of five does.

Models run as Exasol UDFs; search and matching are SQL.

STEP 0 · BUTTON: COMBINE

Five systems, lined up minute by minute



5.3 s

4.5 M rows from 5 systems into 2.0 M aligned minutes

Exasol maps five machine IDs, two clocks (UTC and India time), °C and K, and 15-minute energy blocks that arrive hours late, then fills short gaps.

White = measured. Yellow = filled from the reading before.

STEPS 1–4 · BUTTONS: BEARING · COOLANT · OVERSTRAIN · DRIVE

Each fault has its own pattern across five streams

Fault	Pattern	Found	Came true	Warned
Bearing	vibration ramps up alone	41×	63%	26 of 35
Coolant	temperature gap collapses	44×	50%	22 of 26
Overstrain	torque and power climb	37×	100%	29 of 31
Drive	power drifts, torque normal	56×	54%	27 of 34



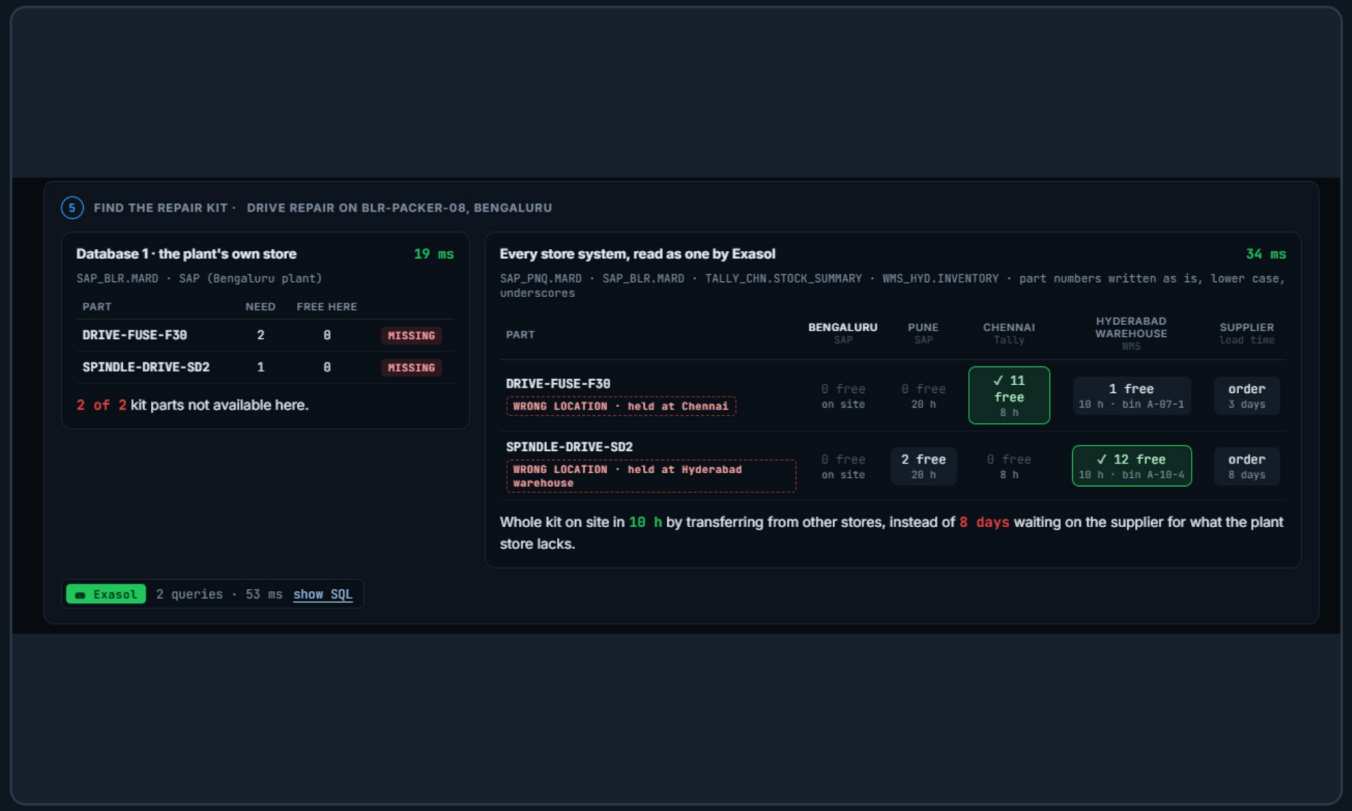






STEP 5 · RUNS ONCE THE FAULT IS SELECTED

The part exists, just not where the machine is



10 h

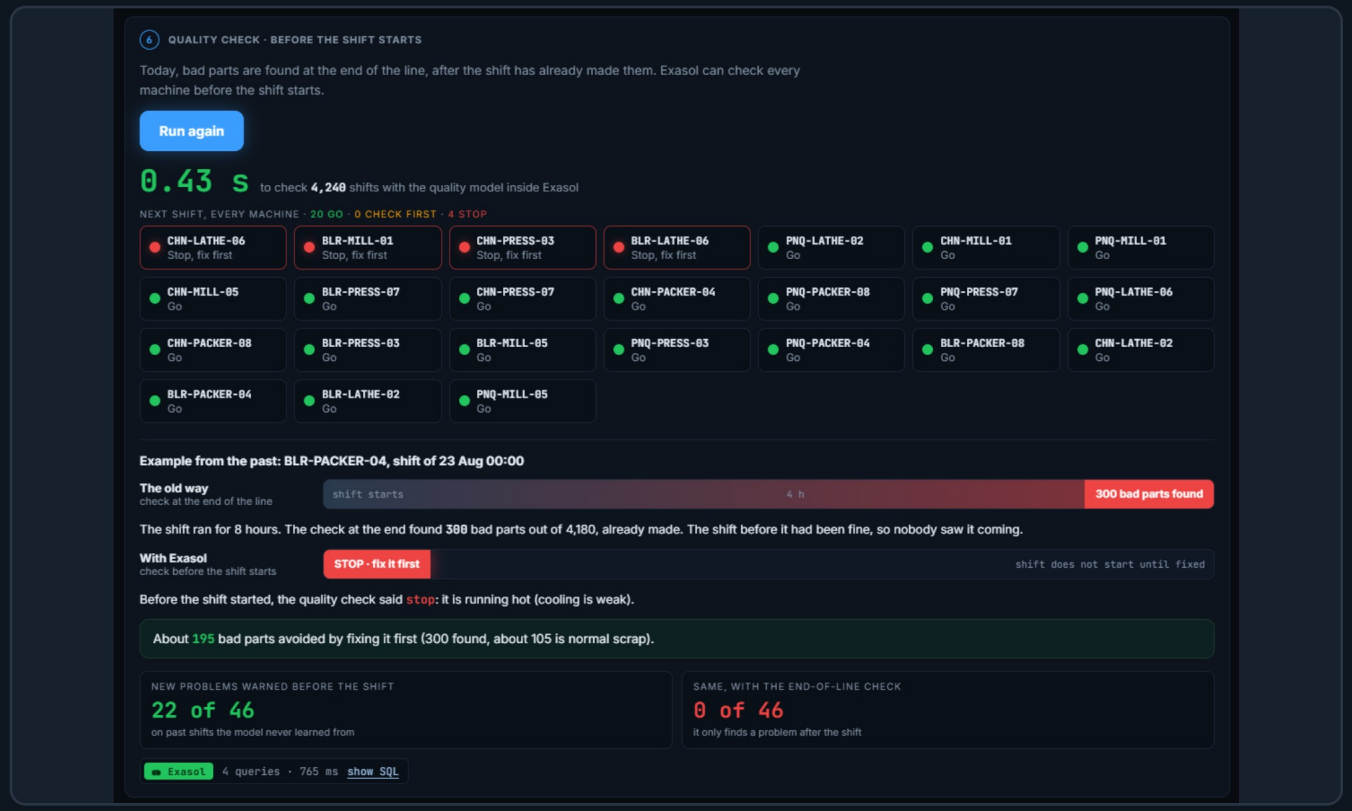
whole kit on site, instead of 8 days waiting on the supplier

Plant store (Bengaluru): both parts missing. Then four store systems read as one, each with its own layout and part-number style.

Found in Chennai and Hyderabad, flagged WRONG LOCATION.

STEP 6 · BUTTON: RUN QUALITY CHECK

Check the shift before it makes bad parts



22 of 46

new quality problems warned before the shift. End-of-line inspection: 0 of 46.

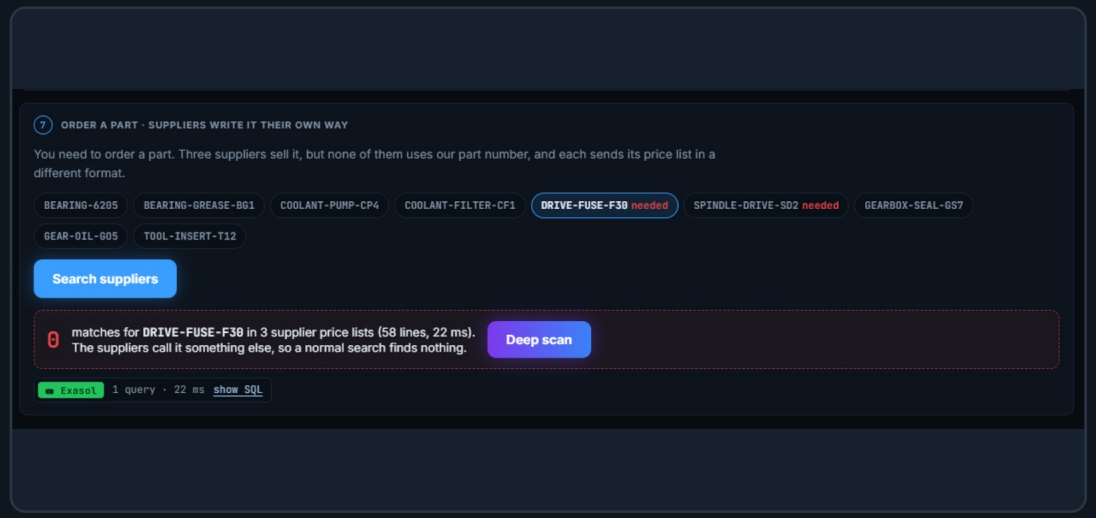
4,240 shifts scored in 0.43 s by a model running inside Exasol as a UDF.

Each machine's next shift: Go, Check first or Stop.

STEP 7 · BUTTONS: SEARCH SUPPLIERS → DEEP SCAN

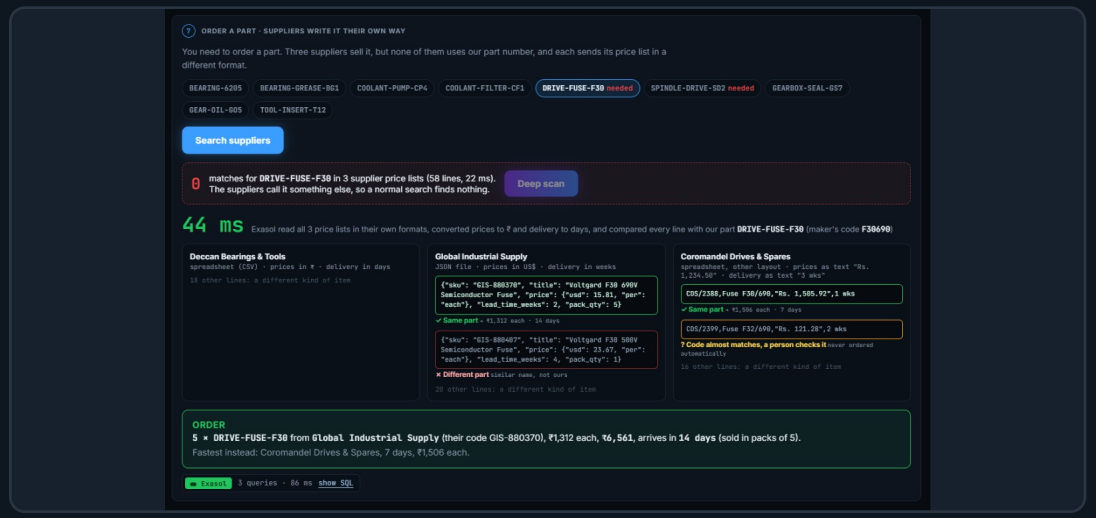
Three suppliers, three formats, none uses our part number

Normal search: 0 found



CSV in rupees and days · JSON in dollars and weeks · CSV with “Rs. 1,234.50” and “1 wks”

Deep scan: order line built



Every line judged SAME, DIFFERENT or ASK. A 6204 bearing next to our 6205 goes to a person, never straight to an order.

Results on held-out data, against a baseline

Step	Result	Compared with
Combine	Model alarms on 9% of hours, 72% precision; warned 32 of 36 stops	Raw sources joined: 97% of hours, 12% precision
Root cause	86% named correctly, on average 17 h before the stop	Guessing the commonest fault: 44%
Parts	Demo kit on site in 10 h	Waiting on the supplier: 8 days
Quality	48% of new problems flagged before the shift	“Last shift was bad” rule: 0%
Suppliers	21 of 21 automatic matches correct, 91% recall	Plain search: 0 matches

Exasol Personal, local on Windows · synthetic plant with known ground truth, 60 days, seed 42 · failure model ROC AUC 0.84, quality model 0.94

How it uses Exasol Personal

Exasol capability	Where it does the work here
In-database AI / UDFs	3 XGBoost models compiled into Python UDFs (FAILURE_RISK, FAILURE_RISK_BATCH, QUALITY_RISK), called from SQL
In-memory MPP	Fusion, pattern search and a numpy SET UDF run across all CPU cores; the bench scales to ~14 M one-minute readings
Analytic SQL	Window functions for fusion and features; EDIT_DISTANCE, JSON_VALUE and REGEXP for supplier matching
Self-tuning	No index, partition or hint anywhere in the SQL
Concurrency	The UI is many small queries; every panel shows its SQL and run time

Why compile the models: no BucketFS upload and no custom container, so it runs on any Exasol with the stock python3 runtime. Parity with XGBoost is tested to 1e-5.

Architecture: everything between source and screen is Exasol

5 sensor systems
vibration · PLC · drive · energy · MES

4 store systems
SAP-style ×2 · Tally-style · WMS

3 supplier lists
CSV · JSON · messy CSV

→

EXASOL PERSONAL

Fusion (SQL) → features (window functions)

FAILURE_RISK UDF → root cause (SQL)

Kit search across 4 stores (SQL)

QUALITY_RISK UDF → Go / Check / Stop

Supplier deep scan → order line (SQL)

→

Demo page + console
FastAPI, plain HTML.
Every panel shows its SQL and run time.

What a real plant would plug in

In the demo (synthetic)	In a real plant	What changes
Vibration, PLC, drive, energy, MES tables	Historian, PLC logs, drives, energy meters, MES	The ID map and the IMPORT; fusion SQL stays
Four store tables	ERP and warehouse systems	One view per system
Three supplier files	Price lists as they arrive	One reader per format
Generated failures	Maintenance history	Retrain; same UDF compiler

One database a plant can install itself: no data team, no platform project.

Run it yourself in about 20 minutes

101
tests, offline, including Exasol reserved-word and alias checks

1
command builds everything: python -m oee.cli all

0
accounts or keys needed; all data generated from a seed

MIT
licence; data sources and tools listed in THIRD_PARTY.md

```
exasol install local → git clone → pip install -r requirements.txt
→ python -m oee.cli all → python -m oee.cli serve → /demo
```

The fault named hours before the stop, the part already on its way.

Five problems. One database. All of it live in Exasol.

github.com/MrinalDesai/exasol-oee-command-center

Mrinal Prakash Desai · ExaThon India 2026 · Track 03: Factories, Fleets & Supply Chains